Akarsh Vyas

Generative AI

Prerequisites

Before starting there are things that you should know.

1. Python Programming
2. Machine Learning Fundamentals
3. Deep Learning Concepts
4. Understanding of Transformers
5. Basic NLP (tokenization, embeddings)

Here’s some important phases before starting.

Phase 1
Research layer

Phase 2
Bedrock models

Phase 3 Providers

Phase 4
This is where you comes in picture

You are going to use these LLM models and create some great AI applications.

Plan of Action

1. LLM foundation
2. RAG system
3. Agentic AI
4. project and deployment

Question 1
what is LLM ?

LLM = Large Language Model
Now break it into 3 words:

1. Large
It is trained on a huge amount of text data
(books, websites, Wikipedia, articles, code, conversations, etc.)

2. Language
It understands and generates human language
Like English, Hindi, Spanish... even programming languages like Python.

3. Model
It is a deep learning system (a neural network) that learns patterns from data.

Question 1
what is LLM ?

When you type:
“What is Python?”

The LLM does NOT:

- Search Google
- Think like a human
- Understand meaning like we do

Instead it:

- Looks at patterns it learned
- Predicts the next most likely word
- Keeps predicting word by word

Some popular LLM’s

GPT Models
Gemini Models
LLaMA
Claude
Grok
Mistral ... etc

You might have seen or heard about these large models.
In Generative AI, we don't build them from scratch
we learn how to use these pre-built LLMs effectively.

But how to use these models?

Open AI

```
from openai import OpenAI

# Create client
client = OpenAI()

# Send request to model
response = client.chat.completions.create(
    model="gpt-4o-mini",  -O
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain what is a Large Language Model in simple terms."}
    ],
)

# Print output
print(response.choices[0].message.content)
```

Gemini

```
import google.generativeai as genai
import os

# Configure API key
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])

# Choose model
model = genai.GenerativeModel("gemini-1.5-flash")

# Send prompt
response = model.generate_content(
    "Explain what is a Large Language Model in simple terms."
)

# Print response
print(response.text)
```

claude

```
import anthropic
import os

client = anthropic.Anthropic(
    api_key=os.environ["ANTHROPIC_API_KEY"]
)

response = client.messages.create(
    model="claude-3-5-sonnet-latest",
    max_tokens=300,
    messages=[
        {"role": "user", "content": "Explain what is a Large Language Model in simple terms."}
    ],
)
print(response.content[0].text)
```

Problem

If every provider has a different SDK,
different syntax,
different response format...
How do we build scalable AI applications?

Langchain is the solution

Components

- Models
- Prompts
- Chains
- Memory
- Indexes
- Agents

Models

Models are the core intelligence layer.
They generate text, embeddings, or other outputs.
They connect your application to LLM providers like:

- OpenAI
- Google DeepMind
- Anthropic

Models

In practical Generative AI, there are mainly 3 important types of models:

Chat / Language Models

1. Generate text
2. Answer questions
3. Write code
4. Summarize content

Embedding Models

1. Convert text into numbers (vectors)
2. Used in search & RAG
3. Help find similar content

Multimodal Models

1. Work with more than just text
2. Can process images, audio, or files

Prompts

A prompt is the instruction we give to a language model.

It tells the model:

- What to do
- How to respond
- What format to use

Without prompts, the model has no direction.

Prompts

LLMs are powerful but they are not mind readers.
The quality of output depends on:
How clearly we instruct them.
Better prompt → Better output.

Prompts

Types of prompts

1. Simple Prompt

Just a direct instruction.
Example:
Explain what is Machine Learning.
Basic. Straightforward.

2. System + User Prompt

Modern chat models use roles:

- System → Sets behavior
- User → Asks question

Example:
System:
You are a helpful AI teacher.
User:
Explain LLMs in simple terms.
System controls tone and style.

Prompts

Types of prompts

3. Prompt Templates

Instead of writing prompts manually every time, we create reusable templates.
Example:
Explain {topic} in simple terms.
Now you can replace {topic} dynamically.
This is useful in applications.

4. Structured output

Used when we want:

- JSON output
- Bullet points
- Fixed format

Example:
Respond in JSON format with keys: definition, example.
This makes output machine-readable.

Chains

A Chain is a sequence of steps connected together.
Instead of:
One Prompt → One Response
We create:
Step 1 → Step 2 → Step 3 → Final Output

Chains

Real AI applications are not one-step.
Example:
User asks:
Summarize this article and translate it into Hindi.
What actually happens?

1. Summarize text
2. Translate summary
3. Return result

That’s a chain.

Memory

Memory allows the AI system to remember past interactions.
Without memory:
Every message is treated independently.
With memory:
The model can use previous context.

Memory

Example:
User: My name is Akarsh.
Later...
User: What is my name?

Without memory → Model doesn’t know.
With memory → It remembers.

Indexes

Indexes allow us to connect external data to an LLM.
LLMs only know what they were trained on.
Indexes help them use:

- PDFs
- Documents
- Databases
- Company data
- Websites

Indexes

LLMs:
❌ Don’t know your company data
❌ Don’t know private documents
❌ Don’t update in real-time

So how do we solve this?
We connect external knowledge.
That system is called Retrieval.

Agents

Agents are AI systems that can decide what action to take.
Unlike normal chains:
Chains → Follow fixed steps
Agents → Decide the next step dynamically

Agents

Normal LLM:
User → Prompt → Response
But what if the task requires:

- Searching the web
- Doing calculations
- Calling an API
- Using a database

The LLM alone cannot do that.
It needs tools.

Enough for now
Lets code